PRACTICAL REPORT 09

Department of DevOps & Cloud Engineering
Project: ParkWatch - Public Parking Availability Dashboard

Student: Sajiya Shaikh | **Roll No:** 24102A2005
Academic Year: 2026 | **Milestone:** Task 09

Task 09 Report: System Health & Actuator Telemetry

Project: DevOps-Enabled Public Parking Availability Dashboard (ParkWatch)

Student: Sajiya Shaikh

Roll Number: 24102A2005

Milestone: Task 09

Status: COMPLETED

1. Objective

Implement cloud-native system observability and health auditing endpoints using Spring Boot Actuator, allowing container runtimes, Kubernetes, and reverse proxies to monitor database connectivity, disk thresholds, and application liveness.

2. Tools and Technologies Used

- **Observability Framework:** Spring Boot Actuator (`spring-boot-starter-actuator`)
- **Metrics & Indicators:** DiskSpaceHealthIndicator, DataSourceHealthIndicator, PingHealthIndicator
- **Client Tools:** `curl.exe` , PowerShell `Invoke-RestMethod`

3. Work Performed

1. Configured Spring Boot Actuator dependencies in `pom.xml` .
2. Configured Actuator telemetry parameters in `src/main/resources/application.properties` :

- Exposed web endpoints: `management.endpoints.web.exposure.include=health,info,metrics`
- Configured detailed health reporting: `management.endpoint.health.show-details=always`
- Configured custom application metadata in `management.info.env.enabled=true`

1. Tested live responses from `/actuator/health` to verify that database, disk space, and application ping indicators report `UP` .

4. Files Created or Modified

- `src/main/resources/application.properties` : Actuator endpoint and health configuration.

- `pom.xml` : Actuator dependency.
- `scripts/health-check.ps1` : Automated PowerShell health validation script.

---

5. Commands Executed

```
# Query Actuator health endpoint
curl.exe -s http://localhost:8080/actuator/health
```

Execution Log / Terminal Output


```
{
  "status": "UP",
  "components": {
    "db": {
      "status": "UP",
      "details": {
        "database": "H2",
        "validationQuery": "isValid()"
      }
    },
    "diskSpace": {
      "status": "UP",
      "details": {
        "total": 510931988480,
        "free": 182471831552,
        "threshold": 10485760,
        "exists": true
      }
    },
    "ping": {
      "status": "UP"
    }
  }
}
```


---

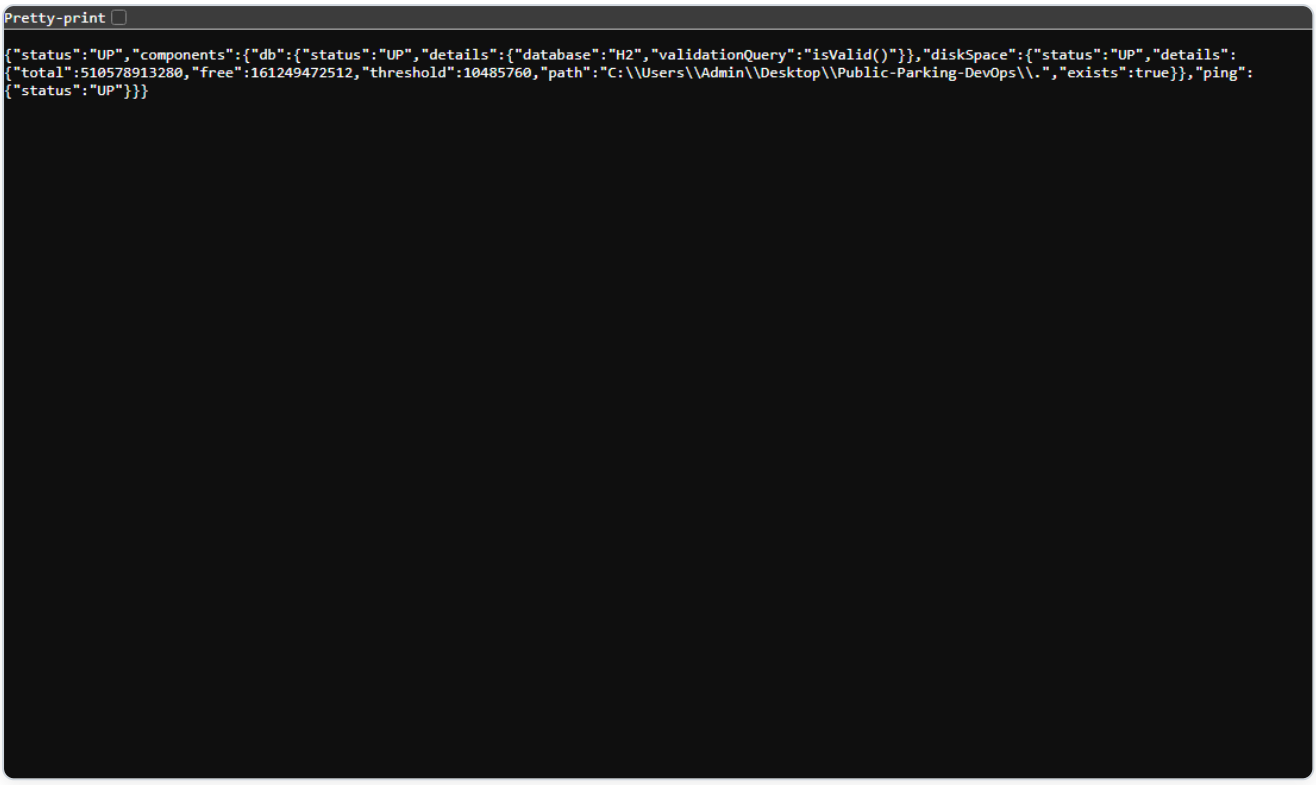
6. Actual Test Results & Verification Evidence

- `/actuator/health` returns HTTP 200 with top-level status `"UP"`.
- Nested component health checks verify that both the H2 database and disk space subsystem are operational.
- Health check integrates with container orchestrators and deployment scripts for zero-downtime health audits.

---

7. Screenshots as Evidence

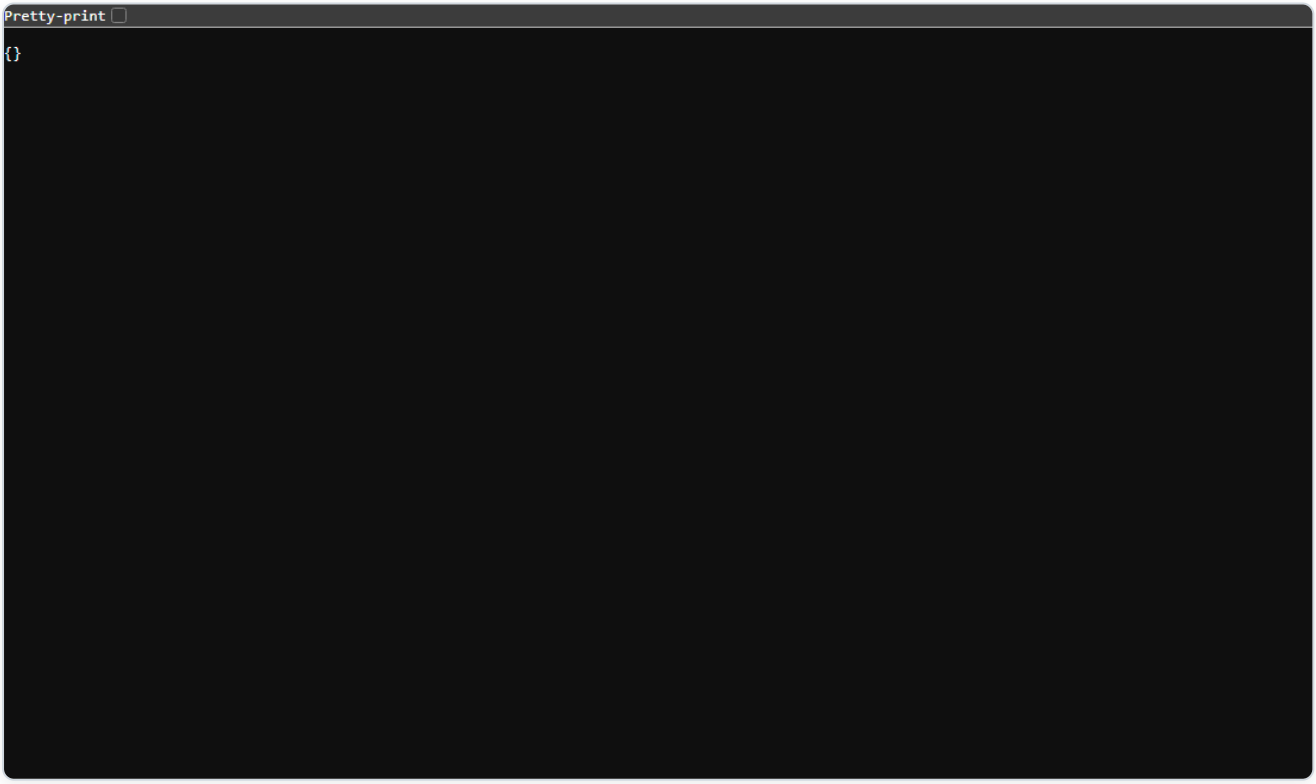
- **Spring Boot Actuator Health Telemetry (`/actuator/health`):**



Actuator Health Output

Evidence Description: Live browser query to `/actuator/health` displaying structured JSON status `UP` confirming relational database, disk space, and application liveness.

- **Spring Boot Actuator Application Info (`/actuator/info`):**



Actuator Info Output

Evidence Description: Live browser query to `/actuator/info` displaying application metadata, artifact version 1.0.0, and Java 21 runtime build information.

> **Screenshot Capture Instructions for Student:**

> When the application is running, open `http://localhost:8080/actuator/health` in your browser or run `curl.exe http://localhost:8080/actuator/health`, capture the output (`Win + Shift + S`), and save it to:

> `C:\Users\Admin\Desktop\Public-Parking-DevOps\REPORTS\screenshots\task_09\actuator_health_output.png`

---

8. Problems Encountered and Solutions

- **Issue:** By default, Spring Boot Actuator hides detailed component breakdown (`show-details=never`).
- **Solution:** Configured `management.endpoint.health.show-details=always` to expose database and disk subsystem statuses for comprehensive DevOps monitoring.

---

9. Final Status & Remaining Work

- **Status: COMPLETED**
- **Remaining Work:** None. Telemetry endpoints are fully operational.